AI Ops Assignment 3: Spark vs. Ray

Kiran Kumar P

The same cleaning pipeline (ingest → cleanse → join → Python UDF → Parquet) was written in PySpark (`spark_clean.py`) and Ray Data (`ray_clean.py`) and run on two Docker clusters of identical size on my laptop. Every number below comes from the logs of those runs; the screenshots are in Section 8.

1. Setup

- **Host:** AMD Ryzen 7 8840HS (8 cores / 16 threads), 14 GiB RAM, Ubuntu, Docker 29.1.3, Compose 2.40.3.
- **One image for both clusters:** the course `docker-spark-cluster` Dockerfile with `eclipse-temurin:17-jre-jammy` as base (`openjdk:17-slim` no longer exists on Docker Hub), Java 17, **Python 3.10.12**, Spark 3.3.4 and Ray 2.59.0 (+ pyarrow 25.0.1, pandas 2.3.3). Both frameworks therefore run the UDF in the same Python interpreter.
- **Data:** NYC TLC Yellow Taxi trips, Jan 2022 – Dec 2024: 36 monthly Parquet files, **1.94 GB, 119,136,044 rows**, plus `taxi_zone_lookup.csv` (265 zones), mounted read-only at `/opt/a3/data` on every node (shared directory).

	Spark standalone (`docker-compose.spark.yml`)	**Ray** (`docker-compose.ray.yml` + `ray start`)
Coordinator + driver	`spark_master`: 2 CPU, 2 GB, Master UI :8080, app UI :4040	`ray_head` (`--num-cpus=0`): 2 CPU, 2 GB, dashboard :8265
Worker A / B	`spark_worker_a/b`: **4 CPU, 3 GB** each	`ray_worker_a/b`: **4 CPU, 3 GB** each
Inside a worker	1 executor, 4 cores, 2 GB heap	4 CPU slots, ≈0.9 GB object store
Cluster total	2 workers, 8 cores, 6 GiB (Master UI)	3 nodes, 8 CPU, 5.17 GiB + 2.22 GiB store
Job launch	`spark-submit --master spark://spark-master:7077`	`python ray_clean.py` on the head (`ray.init(address="auto")`)

Table 1: The two clusters. CPU and memory are hard Docker limits, identical per worker.

Protocol: Spark cluster up → 1-file dry run → 1 warm-up → 3 timed runs → cluster down; then the same for Ray. Only one cluster was running at a time. `monitor_resources.py` sampled `docker stats` (a per-container `top`) once per second for every container during each run.

2. Pipeline and parity

Stage	Spark	Ray Data
Ingestion	read each file, cast to one schema, `unionByName`	`read_parquet` per file + `map_batches(normalize)`, `union`
Cleansing	`dropna()`, `dropDuplicates()`, duration/date/hour, filter	null+NaN mask, hash `repartition(keys)` + per-block `group_by`, same columns, filter
Heavy join	`join(broadcast(zones))` on `PULocationID`	`map_batches(join_with_zones)`, zone table sent to every task
UDF	`F.udf(avg_speed_mph)` (row-at-a-time, pickled)	`map_batches(add_speed_column)`, same function per row
Export	`write.parquet` (200 files, 3.7 GB)	`write_parquet` (200 files, 4.1 GB)

Table 2: Same rules in both, all defined once in `pipeline_common.py` (schema, thresholds, UDF, output columns).

- **Rules:** drop rows with any null/NaN; drop exact duplicate rows; `trip_duration_sec` = dropoff − pickup in whole seconds; keep `duration > 0` and `trip_distance > 0`. The UDF `avg_speed_mph(distance, duration)` = distance / (duration / 3600) is plain Python, so both engines execute identical code. Output: 26 columns.
- **Timing:** every stage ends with a forced materialisation (`cache()+count()` in Spark, `materialize()` in Ray), so lazy evaluation cannot move work into another stage and the UDF is timed on its own. Both pay this cost equally.
- **Parity (`compare_outputs.py`):** both outputs have **110,873,821 rows** (8,262,223 = 6.94% removed), the same 26-column schema, and are identical value by value after sorting, in 24 `pickup_hour` buckets (largest speed difference **0.0**, tolerance 10^{-9}). The SHA-256 of the sorted row hashes is the same on both sides: `f5e32f09...e6b768`.

3. Results (3 timed runs each, warm-up excluded)

Metric	Spark	Ray	Faster
Ingestion (s)	73.2 ± 4.7	40.3 ± 8.6	Ray 1.8×
Cleansing (s)	163.9 ± 17.1	179.2 ± 3.9	Spark 1.09×
Join (s)	78.7 ± 2.4	88.0 ± 6.6	Spark 1.12×
UDF (s)	121.3 ± 0.9	108.9 ± 16.0	Ray 1.11×
Export (s)	73.4 ± 3.8	26.3 ± 1.4	Ray 2.8×
Total wall-clock (s)	**510.4 ± 27.6**	**442.8 ± 20.0**	**Ray 1.15×**
Runs (s)	478.5, 526.1, 526.5	465.5, 427.9, 435.0	
Peak worker CPU (% of one core, limit 400)	427	420	
Peak worker memory (MiB, limit 3072)	2,845	3,072	
Driver container peak (CPU %, MiB)	204, 892	202, 1,418	
Worker processes OOM-killed and retried	0, 0, 0	14, 16, 14	

Table 3: Mean ± sample std over run1–run3. Warm-ups: Spark 443.7 s, Ray 474.9 s. CPU above 400% is a `docker stats` sampling artefact of the CFS quota window.

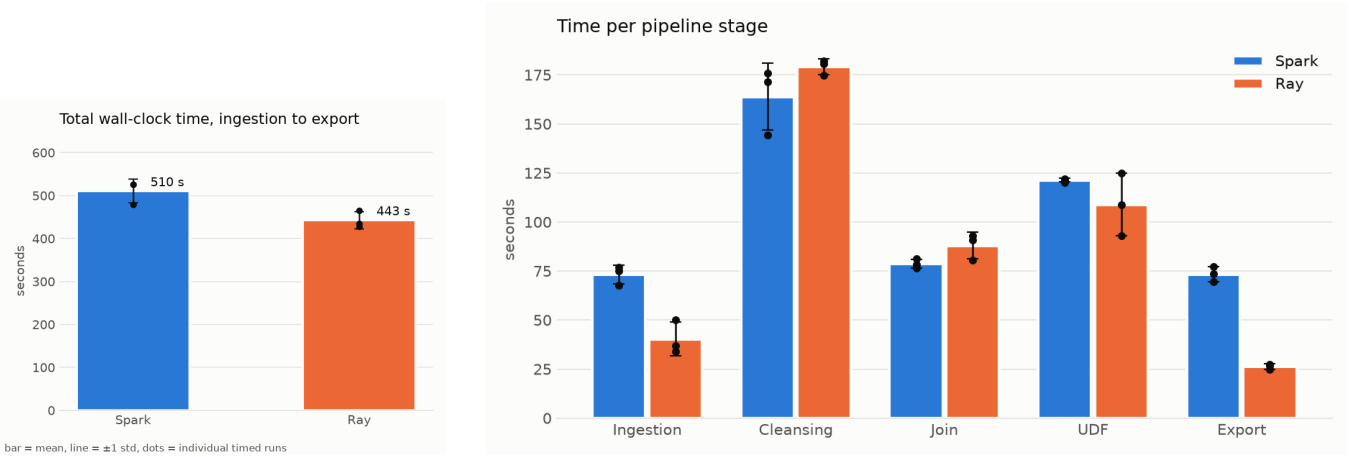


Figure 1: Total time and time per stage (bar = mean, line = ±1 std, dots = individual runs).

- **Where Ray wins:** ingestion and export, the two stages that are pure Arrow I/O. Ray hands Arrow tables from `pyarrow` straight into its object store and back to `pyarrow.parquet`. Spark converts columnar Parquet into its internal row format on read (and back on write), and commits its 200 output files through the Hadoop output committer (write to `_temporary`, then rename); I did not profile which of these accounts for most of the 47 s export gap.
- **Where Spark wins:** cleansing (deduplication = a shuffle of all 119 M rows) and the join. Spark's shuffle is mature, compressed and spills sorted runs to disk (2 GB "Disk Used" per executor in the Executors tab). Ray had to push the materialised dataset through a 2.2 GiB object store: the raylets' cumulative spill counters passed 64 GiB per worker during the session, and every run had 14–16 worker processes killed at the 95% memory threshold and retried.
- **Resources:** both kept all 8 cores busy (peaks ≈400% per worker). Spark stayed 7% under the memory limit because the JVM heap is fixed and the rest spills; Ray sat on the 3 GB limit for most of the run.

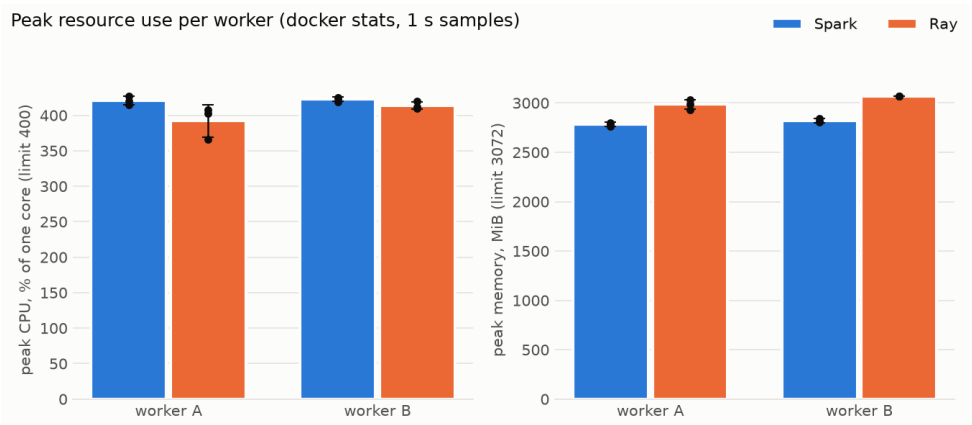


Figure 2: Peak CPU and memory per worker container (1 s `docker stats` samples, 3 timed runs).

– **Variance:** Spark's later runs were 48 s slower than run1, Ray's run1 38 s slower than run2. This is a laptop with a desktop session and browser open (used for the screenshots), so differences under ≈10% between single stages should not be over-read. The total gap (68 s, 13%) is larger than either framework's spread.

4. UDF deep-dive: JVM-based Spark vs. Python-native Ray

What happens per row. In Spark the data lives in the JVM. The plan for the UDF stage (screenshot 08) is `InMemoryTableScan` → `BatchEvalPython` → `Project`: `BatchEvalPython` converts each row to Java objects, pickles batches of 100 rows, writes them over a local socket to a separate `python` worker process, which unpickles them, calls the function, pickles the results and sends them back to the JVM to be converted again. In Ray the worker *is* a Python process: the block is already an Arrow table in shared memory, `to_pylist()` turns two columns into Python floats, the function runs, and `pa.array()` builds the result column. There is no JVM, no socket and no second process.

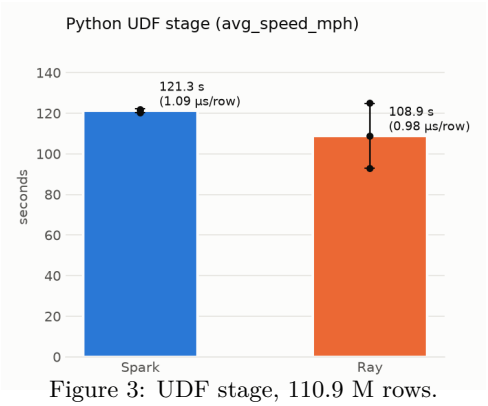


Figure 3: UDF stage, 110.9 M rows.

Measured. Ray's UDF stage was **10% faster** (108.9 vs. 121.3 s; 0.98 vs. 1.09 μs per row of wall-clock, i.e. ≈7.9 vs. 8.7 μs of core time on 8 cores). To see where that time goes I timed the Python side alone on one core in the same image (1 M real rows):

Python-side work per row (1 core)	ns/row	
`avg_speed_mph` only	93	the UDF itself
Spark path: unpickle batch + UDF + pickle results	150	+57 ns serialisation
Ray path: Arrow → list + UDF + list → Arrow	197	+104 ns conversion

– **The function is not the bottleneck.** 93 ns is about 1% of the 7.9–8.7 μs of core time each engine spent per row. More than 97% of the per-row cost is outside the Python code, i.e. moving data, not running Python.
– **Ray's Python-native advantage is real but sits outside Python.** On the Python side Ray's Arrow-to-list conversion is even slower than Spark's unpickling. What Ray saves is the JVM half: the row conversion to Java objects, the socket transfer and the second process per task. That saving is what makes the stage 10% faster, and it would grow with a heavier UDF or with libraries (NumPy, PyTorch) that can use the Arrow buffers without copying.
– **Ray's cost is memory.** The UDF stage had the widest spread in Ray (93–125 s), because the input blocks were spilled and had to be read back from disk. Spark's UDF stage was the most stable number in the whole benchmark (±0.9 s).
– **Fairness note:** Spark's `pandas_udf` (Arrow batches instead of pickled rows) would remove most of its serialisation cost. I kept a row-at-a-time `@udf` because the assignment asks for a custom Python function and it keeps the per-row Python work identical in both engines.

5. What had to change to make Ray finish

Spark ran the final pipeline with default settings and no failures. Ray Data needed four changes on 3 GB workers:

1. **Ingestion OOM:** each monthly file is one Parquet row group, so one read task holds a whole month. Four per worker exceeded 3 GB → each read task requests 1 GB of Ray's `memory` resource (max 2 per worker).
2. **Deduplication:** Ray Data has no `drop_duplicates`; `groupby(all columns).count()` took 153 s on a single file in a development test (vs. about 5 s for the whole cleansing stage now). Hash `repartition` on (pickup, dropoff, PU, DO) plus a per-block `pyarrow group_by` gives the same exact result (development test: the first file read twice returned exactly the original 2,364,474 rows).
3. **Shuffle deadlock:** the default actor-based hash shuffle reserved ≈17 GiB for its aggregators on a 5 GiB cluster and stalled; `SHUFFLE_V2` (task-based) with 128 MiB input batches completes.
4. **Skewed join:** Ray's shuffle join on `PULocationID` (265 keys, a few zones with 4–5 M trips) put partitions larger than a worker into memory; the log shows 35 OOM task failures and no progress on the last 2 of 36 reduce tasks. Both engines now use a broadcast join, the standard plan for a 265-row lookup table.

6. AI-first vs. BI-first, and the winner

– **AI-first project → Ray.** Feature pipelines in AI projects are full of custom Python (tokenisers, embeddings, model inference in `map_batches`), and Ray runs it in the same process as the data, shares Arrow/NumPy memory with it, and continues into Ray Train and Ray Serve on the same cluster. Here it was faster at exactly the Python and Arrow-I/O stages (UDF −10%, ingestion −45%, export −64%).

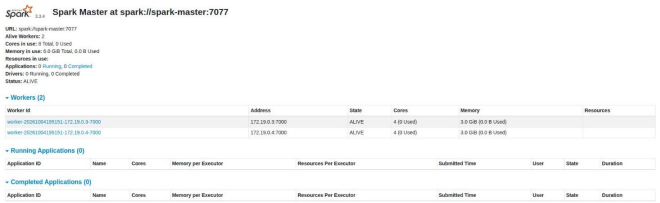
– **BI-first project → Spark.** BI work is joins, aggregations and SQL over large tables. Spark was faster at both shuffle-heavy stages, handled skew and memory pressure by spilling instead of failing, needed no tuning, gave the most predictable times, and has the SQL engine, JDBC/Hive/Delta connectors and BI-tool integration.
– **Winner: Ray, narrowly, for this pipeline.** It finished 1.15× faster (442.8 vs. 510.4 s) with identical output. The win depends on its Python-native I/O and UDF path, and it came at the cost of four non-default settings and running at the memory limit with retried tasks. With less memory per worker, or more joins and aggregations, I would expect Spark to win.

7. Performance Tuning Note (AI-assisted)

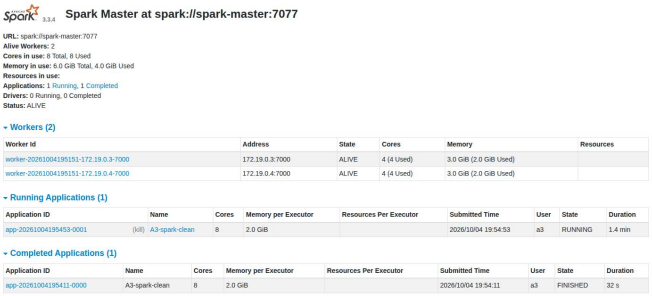
Claude (used as a chat assistant; see `AI_DISCLOSURE.md`) suggested one tuning step in Section 5: hash-repartition + per-block dedup instead of `groupby`. It also explained the Ray out-of-memory errors at ingestion; the fix, a 1 GB memory request per Ray read task, was my choice, as were `SHUFFLE_V2` with 128 MiB shuffle batches, 200 shuffle partitions for both engines (the default of both), and the switch to a broadcast join in both engines after the shuffle join failed. Each change was a response to a failure seen in the logs. None of them changes the output: parity was re-checked after each one. The reported numbers were measured, not generated.

Deviations from the assignment text: the "heavy join" is a broadcast join (Section 5.4); "average speed per hour" is computed per trip in mph, with `pickup_hour` as a column; resources were measured with `docker stats` (the per-container equivalent of `top`) because the Assignment 5 Prometheus/Grafana stack is not available yet; the cluster configuration and network setup were done manually from the course templates (see `AI_DISCLOSURE.md`).

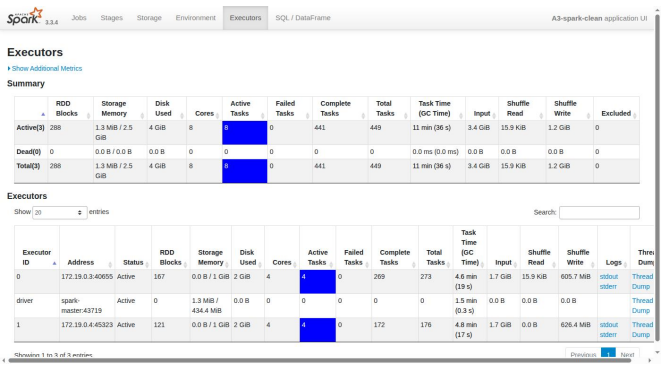
8. Evidence



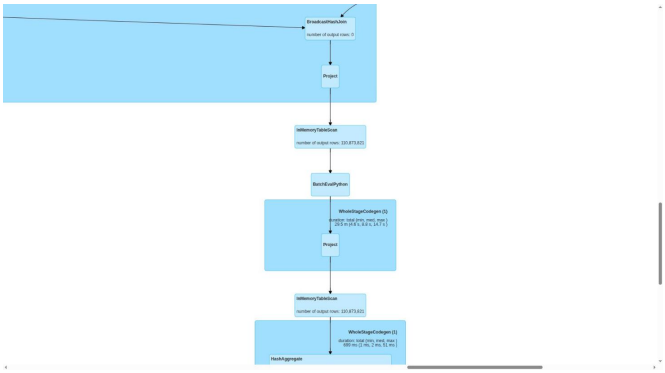
01: Spark Master UI, 2 ALIVE workers, 8 cores, 6 GiB



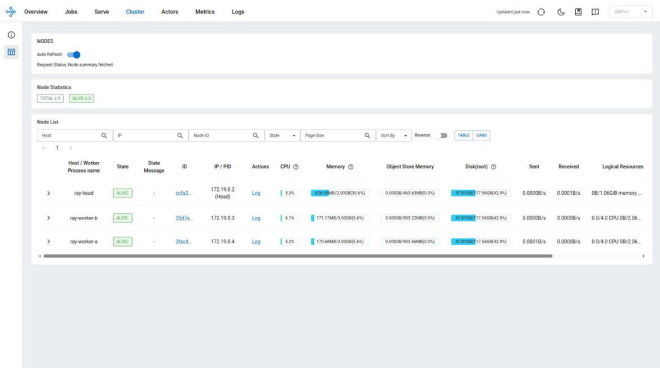
02: application running on both workers



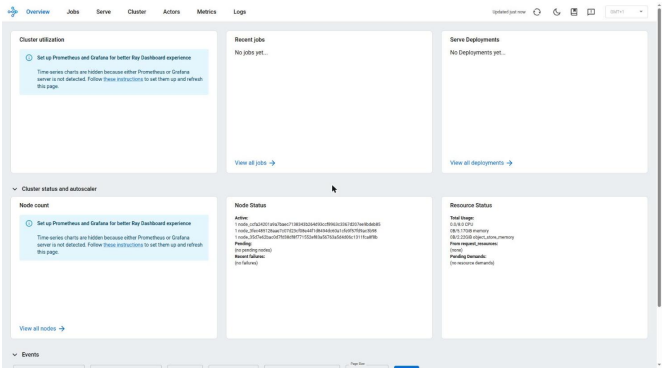
04: Spark UI :4040, one executor per worker



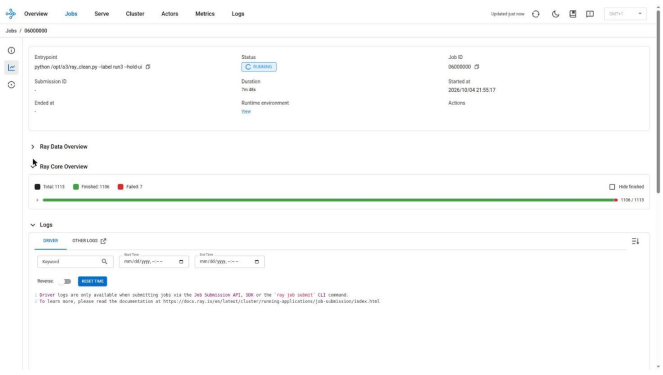
08: UDF query plan, `BatchEvalPython` on 110.9 M rows



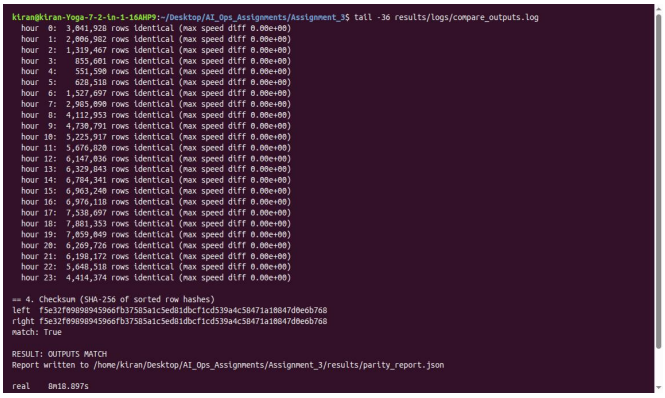
09: Ray Dashboard :8265, head + 2 workers ALIVE



10: Ray cluster resource view, 8 CPU, 5.17 GiB, 2.22 GiB store



14: Ray run3: 1,113 tasks, 7 failed (OOM) attempts, retried



21: parity check, identical rows and checksum

Figure 4: Screenshots taken during the benchmark runs (all 22 are in `screenshots/`). 21 is console output rendered from the parity-check log.